

Lab 14 – (OEL)

Database Application Using Python

Programming Exercise:

Task#1.

Design a database having at least two tables using any DBMS of your own choice.

1. Dept (deptid(pk),dname)
2. Teacher (Tid(pk),name,designation,salary,deptid(fk))
3. Courses (ccode,cname,credithrs,Tid(fk))

In this lab, student will learn how to make a Database App using Python

- Create database connection
- Design GUI of your application according to your own choice using Tkinter or Custom Tkinter Module.
- Write Python code to display data from database into python program.
DML(Insert& Delete)

Task#2.

- Write a Python code to perform insert and delete operation using SQL queries on at least one table mentioned above. When user clicks on delete button , it deletes a particular record and display a message that a record has been deleted successfully.

Source Code:

```
import sqlite3
from tkinter import *
from tkinter import ttk
from tkinter import messagebox

# SQLite database connection
def connect_db():
    conn = sqlite3.connect('university.db')
    return conn

# Create tables in the database
def create_tables():
    conn = connect_db()
    cursor = conn.cursor()

    # Create Dept table
    cursor.execute('''CREATE TABLE IF NOT EXISTS Dept (
        deptid INTEGER PRIMARY KEY AUTOINCREMENT,
        dname TEXT NOT NULL)''')

    # Create Teacher table
    cursor.execute('''CREATE TABLE IF NOT EXISTS Teacher (
        Ttid INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        designation TEXT NOT NULL,
        salary REAL NOT NULL,
        deptid INTEGER,
        FOREIGN KEY(deptid) REFERENCES Dept(deptid))''')

    conn.commit()
    conn.close()

# Display data from the Teacher table
def display_data():
    for item in teacher_table.get_children():
        teacher_table.delete(item)

    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute('''SELECT Ttid, name, designation, salary, deptid FROM Teacher''')
    rows = cursor.fetchall()

    for row in rows:
        teacher_table.insert("", "end", values=row)

    conn.close()

# Add a new teacher
def add_teacher():
    try:
        conn = connect_db()
        cursor = conn.cursor()
        cursor.execute("INSERT INTO Teacher (name, designation, salary, deptid) VALUES (?, ?, ?, ?)",
            (var_teacher_name.get(), var_teacher_designation.get(),
             float(var_teacher_salary.get()), int(var_teacher_deptid.get())))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", "Teacher added successfully")
        clear_inputs()
        display_data()
    except Exception as e:
        messagebox.showerror("Error", f"Failed to add teacher: {e}")
```

```

# Delete a teacher
def delete_teacher():
    try:
        selected_item = teacher_table.selection()
        if not selected_item:
            messagebox.showwarning("Select Record", "Please select a record to delete")
            return

        tid = teacher_table.item(selected_item)["values"][0] # Get Tid from the selected record

        conn = connect_db()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM Teacher WHERE Tid = ?", (tid,))
        conn.commit()
        conn.close()
        messagebox.showinfo("Deleted", f"Teacher with ID {tid} deleted successfully")
        display_data()
    except Exception as e:
        messagebox.showerror("Error", f"Failed to delete teacher: {e}")

# Clear input fields
def clear_inputs():
    var_teacher_name.set("")
    var_teacher_designation.set("")
    var_teacher_salary.set("")
    var_teacher_deptid.set("")

# GUI Design
app = Tk()
app.title("Teacher Management System")
app.geometry("800x500")
app.configure(bg="#f0f0f0")

# Variables for entry fields
var_teacher_name = StringVar()
var_teacher_designation = StringVar()
var_teacher_salary = StringVar()
var_teacher_deptid = StringVar()

# Header Section
header_label = Label(app, text="Teacher Management System", bg="#3498db", fg="white", font=("Arial", 20, "bold"))
header_label.pack(side=TOP, fill=X)

# Input Frame
input_frame = Frame(app, bg="lightgray", pady=10, padx=10)
input_frame.pack(side=LEFT, fill=Y, padx=20)

# Input Fields
Label(input_frame, text="Teacher Name:", bg="lightgray", font=("Arial", 12)).grid(row=0, column=0, sticky=W, pady=5)
Entry(input_frame, textvariable=var_teacher_name, font=("Arial", 12)).grid(row=0, column=1, pady=5)

Label(input_frame, text="Designation:", bg="lightgray", font=("Arial", 12)).grid(row=1, column=0, sticky=W, pady=5)
Entry(input_frame, textvariable=var_teacher_designation, font=("Arial", 12)).grid(row=1, column=1, pady=5)

Label(input_frame, text="Salary:", bg="lightgray", font=("Arial", 12)).grid(row=2, column=0, sticky=W, pady=5)
Entry(input_frame, textvariable=var_teacher_salary, font=("Arial", 12)).grid(row=2, column=1, pady=5)

Label(input_frame, text="Dept ID:", bg="lightgray", font=("Arial", 12)).grid(row=3, column=0, sticky=W, pady=5)
Entry(input_frame, textvariable=var_teacher_deptid, font=("Arial", 12)).grid(row=3, column=1, pady=5)

```

```

# Buttons
Button(input_frame, text="Add Teacher", command=add_teacher, bg="#3498db", fg="white", font=("Arial",
12)).grid(row=4, column=0, pady=10, colspan=2)
Button(input_frame, text="Delete Teacher", command=delete_teacher, bg="red", fg="white", font=("Arial",
12)).grid(row=5, column=0, pady=10, colspan=2)
Button(input_frame, text="Clear Fields", command=clear_inputs, bg="blue", fg="white", font=("Arial",
12)).grid(row=6, column=0, pady=10, colspan=2)

# Table Frame
table_frame = Frame(app)
table_frame.pack(side=RIGHT, fill=BOTH, expand=True, padx=20, pady=10)

# Treeview Table
columns = ("ID", "Name", "Designation", "Salary", "Dept ID")
teacher_table = ttk.Treeview(table_frame, columns=columns, show="headings")
teacher_table.heading("ID", text="ID")
teacher_table.heading("Name", text="Name")
teacher_table.heading("Designation", text="Designation")
teacher_table.heading("Salary", text="Salary")
teacher_table.heading("Dept ID", text="Dept ID")

teacher_table.column("ID", width=50, anchor="center")
teacher_table.column("Name", width=150, anchor="center")
teacher_table.column("Designation", width=100, anchor="center")
teacher_table.column("Salary", width=100, anchor="center")
teacher_table.column("Dept ID", width=50, anchor="center")

teacher_table.pack(fill=BOTH, expand=True)

# Initialize Database and Display Data
create_tables()
display_data()

# Run the App
app.mainloop()

```

Output:

Teacher Management System

Teacher Management System

Teacher Name:
Designation:
Salary:
Dept ID:

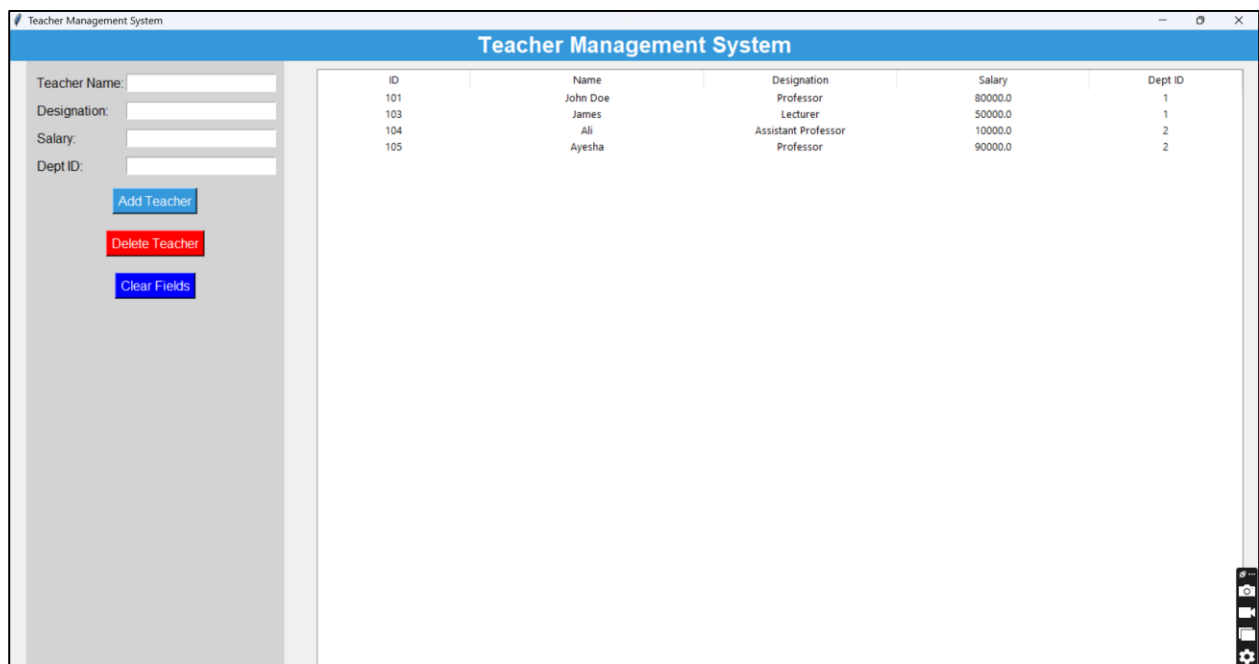
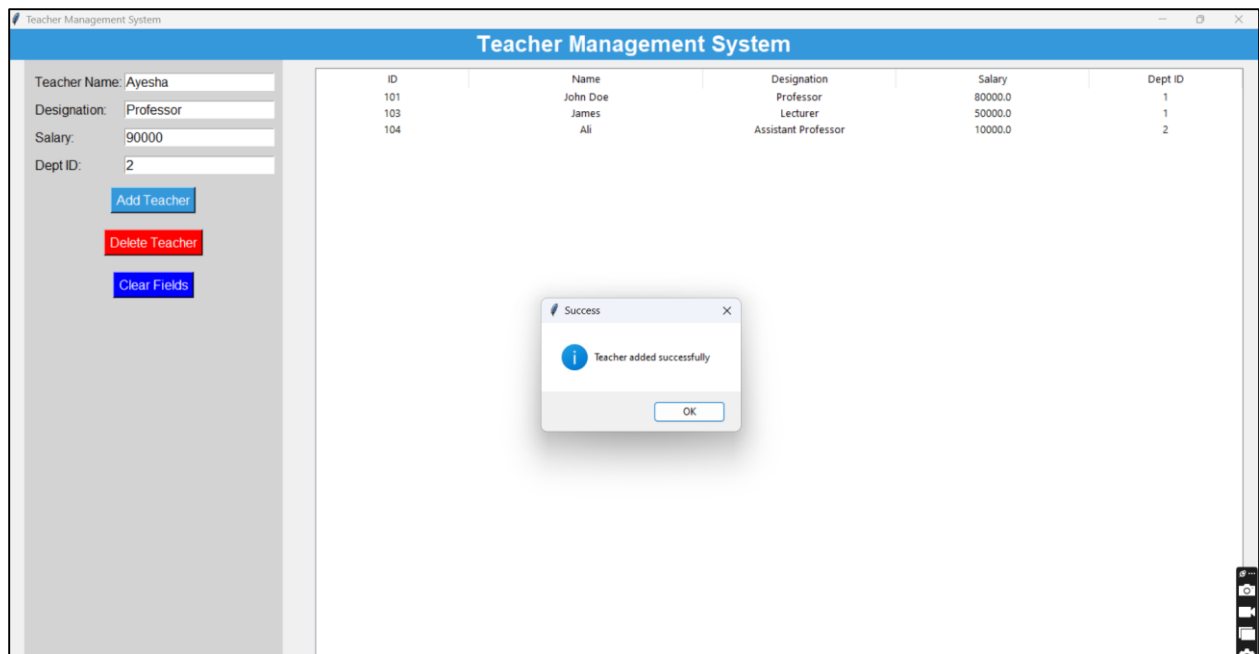
ID	Name	Designation	Salary	Dept ID
101	John Doe	Professor	80000.0	1
103	James	Lecturer	50000.0	1
104	Ali	Assistant Professor	10000.0	2

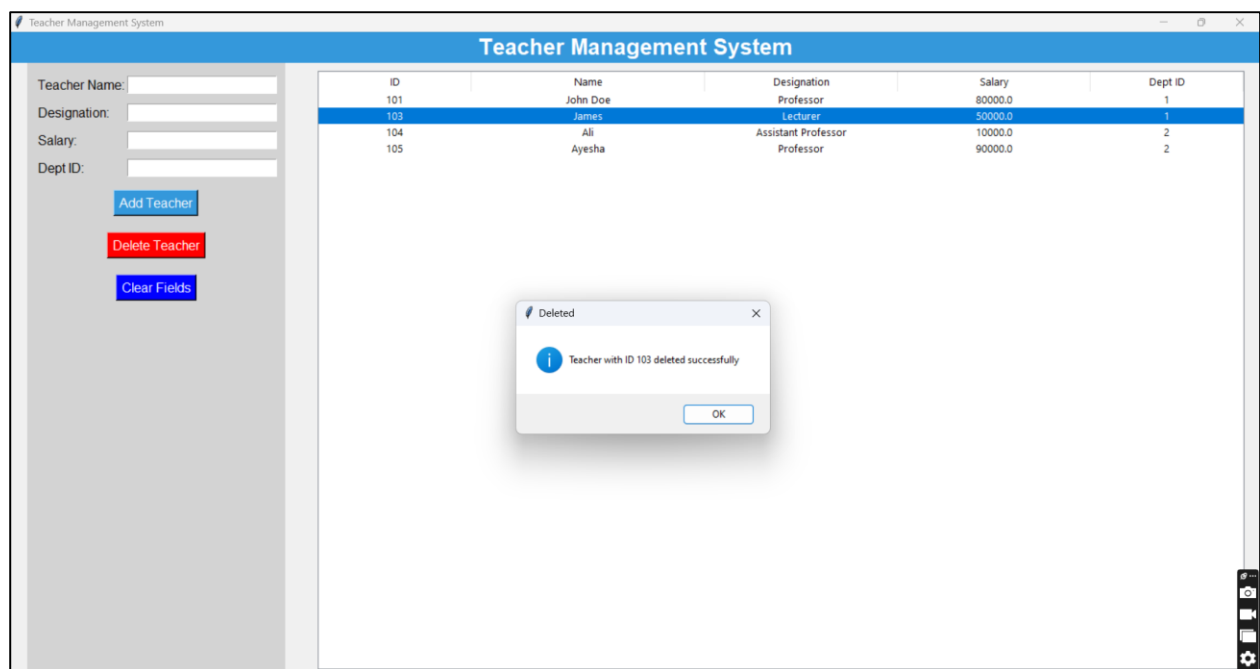
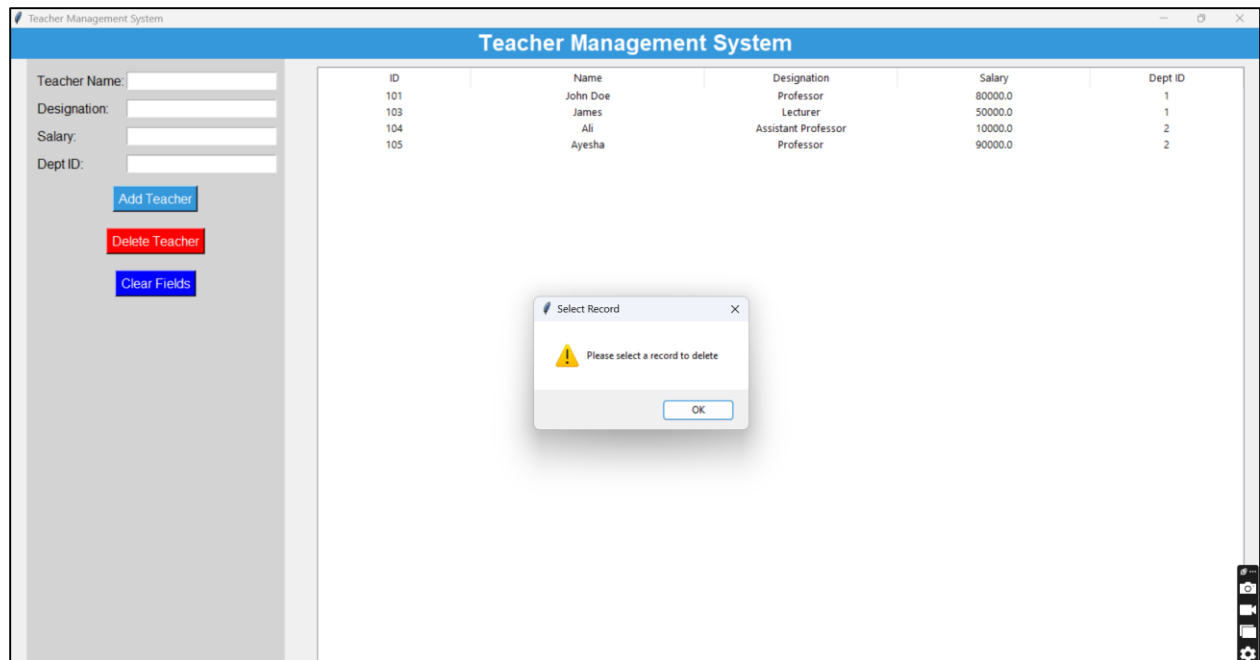
Teacher Management System

Teacher Management System

Teacher Name:
Designation:
Salary:
Dept ID:

ID	Name	Designation	Salary	Dept ID
101	John Doe	Professor	80000.0	1
103	James	Lecturer	50000.0	1
104	Ali	Assistant Professor	10000.0	2







Teacher Management System

Teacher Name:

Designation:

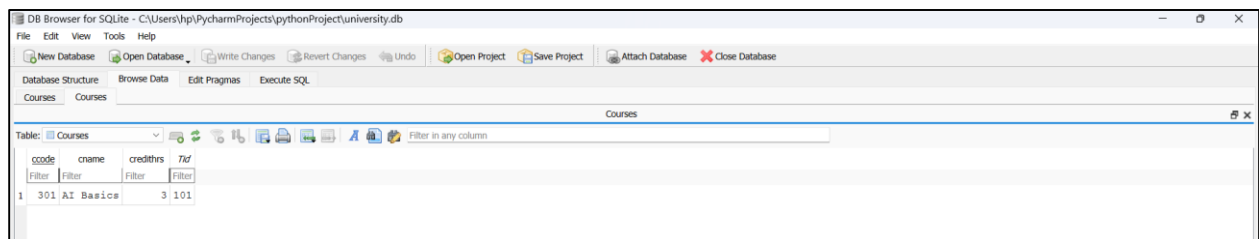
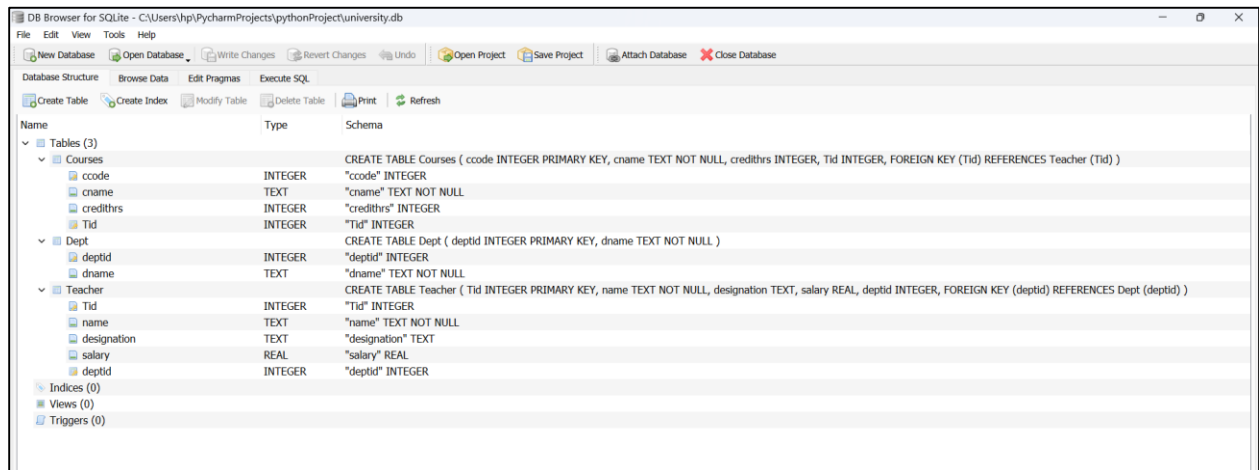
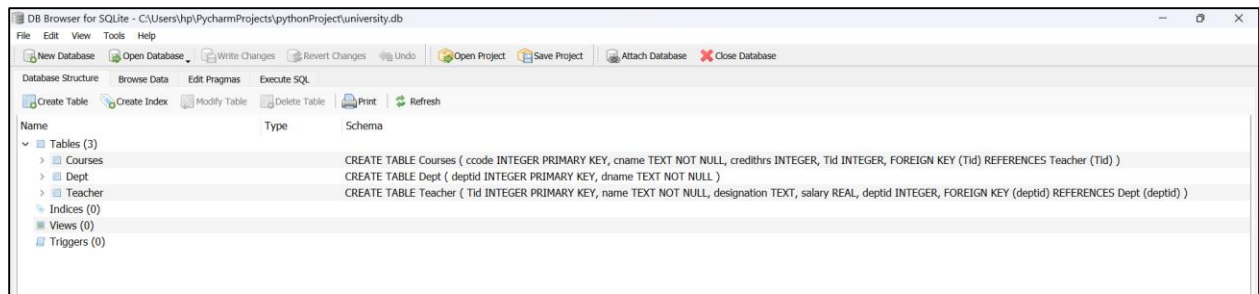
Salary:

Dept ID:

Add Teacher

Delete Teacher

Clear Fields



DB Browser for SQLite - C:\Users\hpi\PycharmProjects\pythonProject\university.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

Courses Courses Dept

Table: Dept

	deptid	dname
1	1	Computer Science
2	2	Mathematics
3	3	Software Engineering
4	4	Software Engineering
5	5	Finance
6	6	

DB Browser for SQLite - C:\Users\hpi\PycharmProjects\pythonProject\university.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

Courses Courses Dept Teacher

Table: Teacher

	tid	name	designation	salary	deptid
1	101	John Doe	Professor	80000.0	1
2	104	Ali	Assistant Professor	10000.0	2
3	105	Ayasha	Professor	90000.0	2